Profiling from Zero

A Complete Hands-On Guide to perf, Nsight, gprof, Valgrind, VTune, and beyond

Prepared for CK | IIT Delhi | Nanopore Project
Prof. Kolin Paul | Summer 2025

Batch 1 · Foundations and perf · Chapters 1 to 5

Contents

CHAPTER 1

Why Profiling Exists

Before we touch a single tool, I want to sit with you for a moment and talk about the thing that makes profiling necessary at all. Because if you do not deeply, viscerally understand *why* profiling exists, then every tool we learn after this will feel like a chore. A flag here, an output table there, some numbers you do not quite trust. That is the experience of most students who pick up perf or Nsight cold. They learn the commands and never learn the question.

The question is older than computers. It is this: **where is my time going?**

I want to tell you a small story to make this concrete. Last year a friend of mine, a very strong systems programmer, spent three weeks rewriting a sorting routine in his data pipeline. He hand-tuned a radix sort. He vectorized with AVX2. He inlined every function. He read papers on cache-oblivious sorting. He was proud of his work. When he finally measured the whole pipeline end to end, the speedup was three percent. Three percent after three weeks. He was crushed. So he profiled. And the profiler told him, in about four seconds, that ninety-one percent of the runtime was spent in a JSON parser at the beginning of the pipeline. The sort he had been agonizing over accounted for two percent of total time. He had been climbing the wrong mountain.

That story is so common it has a name. We call it **premature optimization**, after Knuth, who wrote that premature optimization is the root of all evil. Most people quote that line and stop. The full quote is more useful: "We *should* forget about small efficiencies, say about ninety-seven percent of the time. Premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical three percent." The whole point of profiling is to find that three percent. The two or three functions, the one cache line, the one memcpy that is actually costing you something.

> **Key Insight**
> A profiler is not a tool for making programs faster. A profiler is a tool for telling you the truth. What you do with that truth is optimization. What you do *without* that truth is gambling.

1.1 The Cardinal Sin

I want to name the wrong instinct very directly, because almost every programmer I have ever taught starts with it. Your first instinct, when something is slow, is to *look at the code* and *guess*. You read the function, your eye lands on a loop, you say to yourself "ah, this loop runs a million times, it must be the bottleneck." Maybe you are right. Usually you are wrong. Always you are guessing.

Modern computers are deeply, surprisingly bad at being predicted. A loop that runs a million times might finish in three milliseconds if its data lives in L1 cache. The same loop might take two seconds if the data lives in DRAM. The source code looks identical. The difference is invisible to

your eye but obvious to a hardware counter. This is the central reason profiling exists at all. The code you write is not the work your CPU does.

Think of it like this. Imagine you run a restaurant and customers complain that orders are slow. You could stand in the kitchen and watch the chef. You see him chopping onions, frying eggs, plating food. He looks busy. You conclude the chef is slow. Then someone else points out that orders take twenty minutes not because the chef is slow but because the waiter, on every order, walks across the street to a different building to fetch ketchup. The chef was fine the whole time. The waiter was the bottleneck. You could not see this by watching the chef. You had to step back and measure the entire path of a single order, from sit-down to served, with a stopwatch. *That* is profiling.

The cardinal sin of performance work is this: trying to make something faster without first measuring where time is being lost. Every time you violate that rule, you risk being my friend with his three-week radix sort. Every time you obey it, you tend to find that the real bottleneck is something you would never have guessed.

1.2 What a Profiler Actually Measures

Now we can talk mechanics. A profiler is not magic. It is a program that reads numbers. Those numbers come from a place I want you to know about from day one: the **Performance Monitoring Unit**, or PMU. The PMU is a small block of dedicated hardware inside every modern CPU. It exists for one reason: to count events. Not to compute. Not to run instructions. Just to count.

Inside the PMU there are a small number of physical counters. On a typical Intel Skylake-class chip there are four general-purpose counters and three fixed counters per core. These are real silicon registers. You can program each one to count a specific kind of event. Number of cycles that elapsed. Number of instructions that retired. Number of L1 cache misses. Number of branch mispredictions. Each event has a numeric code. You write the code into a configuration register, the counter starts incrementing, and at any moment you can read its value.

When you run *perf stat* on a program, what happens under the hood is roughly this. The kernel asks the PMU to count a set of events for the duration of your program. The CPU runs your code as it always does. The counters tick up. When your program exits, perf reads the final values out of the PMU and prints them. The cost of this is essentially zero. The counters were going to tick whether or not perf was looking. You are simply observing what the silicon was already doing.

> **Beautiful Detail**
> The PMU is one of the only places in software engineering where you get truth for free. No probe effect. No instrumentation overhead. The CPU was always counting. The profiler just reads the meter.

Other profilers work differently. Some interrupt your program a thousand times a second and ask *where are you right now?* That is sampling. Some rewrite every instruction in your binary to add bookkeeping. That is instrumentation. Some, like Nsight on GPUs, read GPU-side counters that work the same way as the CPU PMU but live on a different chip. We will see all of these. The point right now is that the underlying philosophy is always the same: the profiler is a measurement

device, the program is what is being measured, and the data is closer to truth than your intuition will ever be.

1.3 The Four Questions Every Profiler Answers

Every profiling session you will ever run, for the rest of your career, comes down to one of four questions. I want to put them in front of you now, because as soon as you have these four questions in mind, the entire ecosystem of tools stops feeling like a zoo and starts feeling like a toolbox.

Question 1: Where is time spent?

This is the simplest and the most common. You have a program. It is slow. You want to know which functions are eating the clock. This is called **hotspot analysis**. The tool of choice is usually a sampling profiler like perf record, gprof, or Nsight Systems. The output is a sorted list: this function consumed forty percent of total time, that function fifteen percent, and so on down the long tail. You look at the top of the list, and that is where you point your optimization efforts.

For our project specifically, this question becomes: in Dorado, which CUDA kernels are taking the most GPU time? In Kraken-2, which C function is consuming the most cycles? Those two answers alone will tell us where to focus.

Question 2: Why is time spent there?

Hotspot analysis tells you *where*. It does not tell you *why*. A function might be slow because it does a lot of arithmetic, in which case the answer is "reduce arithmetic." Or it might be slow because it stalls waiting for memory, in which case the answer is "improve cache locality." Or because of branch mispredictions. Or because of synchronization. The why determines the fix. Getting the why wrong wastes weeks.

This is called **bottleneck analysis**, and it requires hardware counters. perf stat will tell you the IPC. perf record with an event filter will tell you which lines miss the cache. Nsight Compute will tell you whether a GPU kernel is compute-bound, memory-bound, or latency-bound. We are going to spend a lot of time on this question, because it is the question most beginners cannot answer, and it is the question that separates good profiling from cargo-cult profiling.

Question 3: How much faster could it be?

This question is the most underappreciated. Suppose your matrix multiplication takes ten seconds. Should you be happy or sad? The honest answer is: it depends on the theoretical ceiling. If the hardware can in principle do this computation in two seconds, you have a 5x speedup waiting to be discovered. If the hardware peak is nine seconds, you are already very close to optimal and should stop optimizing.

The framework that answers this is the **Roofline Model**, which we will study in Chapter 5. The idea is simple. Every algorithm has an arithmetic intensity: how many floating-point operations it does per byte of memory traffic. Every machine has a peak compute rate and a peak memory bandwidth. Plot one against the other and you get a piecewise-linear ceiling. Your program lives

somewhere below that ceiling. The gap is your optimization headroom. The roofline is the difference between optimizing forever and knowing when to stop.

Question 4: What changed?

Performance regresses. You add a feature, you upgrade a library, you change a compiler flag, and suddenly the code is twenty percent slower and nobody knows why. The fourth question every profiler must answer is: *compared to last week, what changed?* Tools like *perf diff* directly compare two profiling runs and highlight the functions whose share of runtime moved. This question is what makes profiling sustainable rather than a one-time activity.

> **How This Connects to Your Project**
>
> For the Nanopore pipeline, we have a specific question of each kind. Where: which Dorado attention kernel dominates basecalling time, and which Kraken-2 hash function dominates classification time. Why: is the Dorado kernel memory-bandwidth-bound or compute-bound, and what is the L3 miss rate on the Kraken-2 lookup. How much faster: what fraction of the T4 GPU peak bandwidth is Dorado achieving, and what fraction of DRAM bandwidth is Kraken-2 achieving. What changed: when we add the Hot-K-mer LRU cache, does the LLC miss rate drop and by how much.
>
> Every chapter from here on will give you one or more tools that answer one or more of these four questions. You should always know, when you run a tool, which of the four questions you are trying to answer. That habit alone will put you ahead of ninety percent of working engineers.

1.4 The Taxonomy of Profilers

There are dozens of profiling tools in the world. They look bewilderingly different from the outside. They are not. Underneath, every profiler falls into one of four families, distinguished by *how* they obtain measurements. Once you know these four families, every tool is immediately legible.

Family 1: Sampling Profilers

A sampling profiler does not watch your program continuously. It interrupts at a fixed frequency, typically a few thousand times per second, and at each interrupt it records what the program was doing. Specifically it records the program counter, the function that contains that address, and sometimes the full call stack. After your program finishes, the profiler aggregates: function *foo* appeared in forty percent of samples, therefore foo consumed roughly forty percent of CPU time.

The brilliance of sampling is that overhead scales with sampling rate, not with program size. At 99 Hz, the overhead is well under one percent. The cost of being interrupted ninety-nine times a second is negligible. Yet the statistical accuracy is excellent over a multi-second run. perf record, gprof in some configurations, and the Linux kernel's own perf_events framework are all sampling profilers.

The catch with sampling is that it is statistical. If your program runs for only twenty milliseconds and you sample at 99 Hz, you get two samples. That is not enough to know anything. Sampling needs duration. For very short programs you need a different approach.

Family 2: Instrumentation Profilers

An instrumentation profiler modifies your program, either at compile time or at run time, to insert bookkeeping at the entry and exit of every function or every basic block. Now there is no statistics involved. Every single function call is counted exactly. Every cache miss is simulated exactly. The numbers are perfectly accurate.

The cost is overhead. gprof, which inserts call-counting code at compile time when you pass *-pg*, slows your program down by perhaps two times. Valgrind, which rewrites every instruction at runtime, slows your program down by ten to fifty times. Cachegrind, which additionally simulates a virtual cache, can slow you down by a hundred. These tools are not for production. They are for forensic deep dives. When sampling tells you a function is hot but you cannot tell *why*, you turn to instrumentation.

Family 3: Hardware Counter Profilers

I described these in Section 1.2. The PMU counts events. A hardware counter profiler asks the PMU to count specific events for the duration of the program, then reads the totals. perf stat is the canonical example. VTune's hardware event collection is the same idea with more polish. LIKWID is a low-level wrapper around the same underlying mechanism.

Hardware counter profilers have essentially zero overhead. The counters tick whether or not anyone is watching. They are limited only by the number of physical counters the chip provides and by what events the chip vendor chose to expose. They are the workhorse for the *why* question. If you want to know why your function is slow, the answer almost always comes from hardware counters.

Family 4: GPU Profilers

GPUs do not run on the host CPU. They have their own clock, their own memory hierarchy, their own counters. A GPU profiler is a CPU process that talks to GPU-side counters via a vendor-specific runtime. NVIDIA's tools are Nsight Systems, which traces the whole CUDA timeline and shows what kernels ran when, and Nsight Compute, which dives deep into a single kernel's per-cycle behavior. They are conceptually similar to perf record and perf stat, but on the other side of the PCIe bus.

Every modern GPU has a PMU equivalent. NVIDIA calls them SM activity counters, memory throughput counters, warp stall reason counters. They are read in the same spirit as CPU hardware counters: ask the silicon, get the truth, no probe effect.

> **Common Pitfall**
>
> Beginners try to use a sampling profiler when they should be using a hardware counter, or vice versa. Rule of thumb: if you want to know *where*, use a sampling profiler. If you want to know *why*, use a hardware counter. If you cannot trust the sampling result because the program is too short or too noisy, switch to instrumentation. If the program runs on the GPU, none of the CPU tools apply and you need Nsight.

1.5 The Real Pipeline: Dorado and Kraken-2

Let me ground all of this in the work we are actually doing this summer. Our pipeline has two stages. Stage one is **Dorado**, the Oxford Nanopore basecaller. Stage two is **Kraken-2**, the k-mer taxonomic classifier. They have almost nothing in common technically, which is exactly why they will let us exercise the full toolbox.

Dorado takes raw electrical signal from a nanopore sequencer and converts it into nucleotide sequences. The model is a deep neural network. There is a 1D convolutional stem, then a stack of Transformer encoder blocks, then a CTC decoder that produces the final base sequence. Every part of this runs on the GPU. The model weights live in GPU memory. The signal data is streamed in from disk and copied from CPU to GPU. The output bases are copied back. Profiling Dorado means profiling kernels: which CUDA kernel takes the most time, what is its memory bandwidth utilization, what is its SM occupancy, is it compute-bound or memory-bound. For Dorado we will live almost entirely in **Nsight Systems** and **Nsight Compute**.

Kraken-2 is the opposite. It is a CPU program. It loads a 180 GB hash table of k-mer to taxonomy mappings, then for each input read it slides a window across the sequence and looks up each k-mer in the hash. The model is a hash table. The hot loop is a hash function and a memory lookup. Kraken-2 runs on tens of CPU threads. There is no GPU. The bottleneck is almost certainly memory. Each hash lookup touches a random address in a 180 GB table, which means almost every lookup is an L3 cache miss, which means almost every lookup spends a couple of hundred cycles waiting for DRAM. For Kraken-2 we will live in **perf stat**, **perf record**, and **Cachegrind**.

> **How This Connects to Your Project**
>
> Look at what we just did. We took two real workloads and, before running a single profiler, we have already formulated specific hypotheses about each one. Dorado: GPU-bound, kernel-dominated, probably memory-bandwidth-limited on the attention layers. Kraken-2: CPU-bound, hash-lookup-dominated, almost certainly LLC-miss-limited. The job of the profiler from here is not to discover this from scratch. It is to *confirm or refute* our hypothesis and then to give us the exact numbers we need to put in front of Prof. Paul. Good profiling is not exploration. It is hypothesis testing.

1.6 The Two-Page Report We Are Building Toward

Everything in this book is in service of a single deliverable. A two-page profile report for Prof. Paul. By the end of Chapter 19 you will be able to write this report without thinking. But it is worth knowing now what the finished product looks like, so you know what the chapters are giving you tools to fill in.

Page one is Dorado. The page contains a Nsight Systems timeline screenshot showing CPU-GPU coordination. A table of the top five CUDA kernels by time, with columns for duration, SM occupancy, DRAM throughput, and L2 hit rate. A statement of whether the workload is compute-bound or memory-bound, derived from the GPU roofline. A one-sentence recommendation for what to optimize first.

Page two is Kraken-2. The page contains a perf stat table with cycles, instructions, IPC, branch miss rate, and cache miss rate. A perf report of the top five functions by self time. A Cachegrind LLC miss rate for the hash lookup function. A one-sentence recommendation, which we already know in our hearts will be "add a hot-k-mer cache to keep recently used entries in L3." But we will not say that until the numbers say it.

The whole book is the path from here to there. Every tool we learn fills in one cell of that table. Every command we run is a number on that page. Keep that in mind. You are not learning perf for its own sake. You are learning what cells of the report perf populates.

> **What We Just Learned**
>
> The cardinal sin of performance work is optimizing without measuring. The job of a profiler is to tell you the truth about where time is going.
>
> A profiler is a measurement device. Most of them are reading hardware counters in the CPU's Performance Monitoring Unit, which counts events for free.
>
> Every profiling session answers one of four questions: where, why, how much faster, what changed. Knowing which question you are asking determines which tool to reach for.
>
> Profilers come in four families: sampling, instrumentation, hardware counter, and GPU. Use sampling for the where question, hardware counters for the why question, instrumentation for forensic depth, and GPU profilers for anything past the PCIe bus.
>
> Our pipeline has two halves that exercise different tool families. Dorado is a GPU workload for Nsight. Kraken-2 is a CPU memory-bound workload for perf and Cachegrind. Together they cover everything.

> **Before You Move On**
>
> Pause and ask yourself: of the four questions, which one have you historically ignored when working on your own code? Be honest. Most people skip the *why* and the *how much faster*. Both questions are where engineers go from competent to excellent.
>
> Next chapter is the only chapter in the book with no tool in it. It is pure CPU architecture: cache hierarchies, branch prediction, IPC, the things the PMU is counting. If you skip Chapter 2 you will be reading numbers without knowing what they mean. Do not skip it.

CHAPTER 2

How CPUs Execute Code

Now we slow down and look at the machine. This is the only chapter in the book with no tool in it. We are going to talk about how a modern CPU actually executes code, because every number a profiler ever shows you is a number about something in this chapter. If perf tells you the cache miss rate is forty-seven percent, you need to know what a cache is, why misses are expensive, and how forty-seven percent relates to your runtime. You cannot read profiler output without first reading the chip.

Most CS students have a mental model of CPUs that was true in 1985. They think a CPU is a thing that fetches an instruction, decodes it, executes it, writes back the result, and moves on to the next one. One instruction in, one cycle later, one result out. This model has not been true for almost forty years. It survives because it is what introductory architecture courses teach, and because reading machine code looks exactly like that model. The code is sequential. The execution is anything but.

I am going to walk you through what is actually happening, layer by layer, until by the end of this chapter you can stare at our naive matmul inner loop and tell me, cycle by cycle, where the time is going. Once you can do that, the perf counters in Chapter 3 will read like sentences instead of numbers.

2.1 The Memory Hierarchy Is the Story

I want to start with the single most important fact about modern computing performance, and I want to say it bluntly so it sticks. **Almost every program is bottlenecked by memory, not by computation.** Not arithmetic. Not branches. Memory. The CPU is, in the literal cycle-counting sense, sitting around waiting for DRAM far more often than it is doing work.

Here is why. A modern CPU running at 3 GHz can execute roughly four arithmetic instructions per cycle on each of perhaps eight cores. That is 96 billion arithmetic operations per second per chip. Meanwhile, main memory delivers data at roughly 30 GB per second on a typical desktop. If your arithmetic operates on 8-byte doubles, the memory bandwidth limit is 30 / 8 = 3.75 billion operands per second. The compute capacity exceeds the memory bandwidth by a factor of 25. The chip is, in pure throughput terms, twenty-five times faster at math than at fetching the numbers it needs to math on.

So how does this not bring everything to a halt? Caches. The CPU has several layers of small, fast memory that sit between the registers and the DRAM. When you access an address, the cache keeps a copy nearby in case you ask for it again. If you do, the answer comes back fast. If you do not, you pay the full DRAM round-trip. This single mechanism, and how well your program plays with it, determines almost everything about performance.

The Hierarchy in Numbers

Let me put real numbers on the hierarchy. These are typical values for a modern x86 chip. They vary across CPUs but the orders of magnitude are stable.

- **Registers:** capacity around 16 general-purpose 64-bit registers per core, latency 0 cycles, the CPU has them already.
- **L1 cache:** capacity around 32 KB per core, latency 4 to 5 cycles, very fast.
- **L2 cache:** capacity around 256 KB to 1 MB per core, latency 12 to 14 cycles.
- **L3 cache (LLC):** capacity around 8 to 32 MB shared across all cores on the socket, latency 35 to 50 cycles.
- **DRAM:** capacity gigabytes to terabytes, latency 200 to 300 cycles.
- **Disk (NVMe SSD):** capacity terabytes, latency tens of thousands of cycles.
- **Network:** latency hundreds of thousands to millions of cycles.

Let that sink in. The gap between an L1 hit and a DRAM access is fifty times. The gap between L1 and disk is roughly ten thousand times. If your hot loop misses L1 even once in fifty accesses, you spend half your time waiting. If it misses L3 even once in two hundred accesses, you spend half your time waiting.

> **A Useful Analogy**
> Think of memory like fetching things from increasingly far-away storage. A register is the item already in your hand. L1 is on your desk. L2 is in a drawer. L3 is in a closet down the hall. DRAM is in a warehouse across town. Disk is a warehouse on another continent. Every memory access is an errand. The whole game of cache-aware programming is to keep your errands close.

Cache Lines: The Atom of Memory Traffic

Now a fact that is going to come back over and over again. Caches do not fetch single bytes. They fetch **cache lines**. A cache line is a chunk of contiguous memory, almost always 64 bytes on x86, sometimes 128 bytes on Apple Silicon and a few other architectures. Every memory operation between any two levels of the hierarchy is a transfer of one cache line.

This single fact has enormous consequences. Suppose you read a single 8-byte double from a cold address. The CPU does not fetch 8 bytes. It fetches the 64-byte cache line containing those 8 bytes. The other 56 bytes ride along for free. If your next memory access is to one of those neighbor bytes, you get a cache hit and pay nothing. If your next access is to a totally different cache line, you pay a full miss again.

This is the formal reason that *spatial locality* matters. If your program accesses memory in patterns where neighboring bytes get touched soon after each other, the cache pays for itself. If your program jumps around to random addresses, the cache is useless. Every 64-byte transfer brings in 56 bytes that get thrown away.

Look at the inner loop of *matmul_naive*. The inner accumulation reads *A[i*n + k]* and *B[k*n + j]* and writes *C[i*n + j]*. The variable that increments in the innermost loop is *k*. So consecutive iterations

of the inner loop access *A[i*n], A[i*n + 1], A[i*n + 2]*... and *B[0*n + j], B[1*n + j], B[2*n + j]*... and the same *C[i*n + j]* over and over.

Now think about cache lines. *A[i*n + k]* as k increments visits eight consecutive doubles per cache line. Excellent spatial locality. Eight hits for every one miss. *B[k*n + j]* as k increments visits doubles that are *n* apart, which for N=1024 is 8192 bytes apart, which is 128 cache lines apart. Every single iteration triggers a fresh cache line fetch. Zero spatial locality. That column-major access to B is the entire reason naive matmul is slow, and we have not even seen a profiler yet.

The Prefetcher: Free Speed for Sequential Access

Modern CPUs have a hardware prefetcher. The prefetcher watches your access patterns. If it sees you reading address X, then X+64, then X+128, it concludes you are walking forward through memory and it starts fetching X+192 and X+256 before you ask for them. By the time your code requests them, they are already on the way or already in cache.

The prefetcher is essentially free speed, but only for predictable patterns. Stride-1 access (every byte sequential): the prefetcher loves you. Stride-2, stride-4, even fairly large strides: the prefetcher can often keep up. Random access, pointer chasing through a linked list, hash table lookups: the prefetcher is useless. There is no pattern to predict.

This is the single deepest reason that the *i, k, j* loop order in *matmul_ikj* is so much faster than the *i, j, k* order in *matmul_naive*. In the ikj order, both A and B are accessed with stride 1 in the inner loop, and the prefetcher fetches every line before the code asks for it. In the ijk order, B is accessed with stride n, the prefetcher cannot keep up, and every inner iteration eats a cache miss.

> **How This Connects to Your Project**
>
> Kraken-2 lives or dies by the prefetcher question. The whole pipeline is a series of hash table lookups on k-mers. A hash function by design scatters lookups to random addresses across a 180 GB table. The prefetcher cannot predict any of it. Every lookup is independent, every lookup is random, and every lookup probably misses L3. That is exactly what perf is going to confirm in Chapter 3. The Hot-K-mer LRU cache works by keeping the most frequent k-mers in a small contiguous structure that does fit in cache, so the common path no longer pays the random-access tax.

2.2 Branch Prediction

Caches handle memory. Branch predictors handle control flow. Both are about hiding latency that would otherwise stall the pipeline. Let me explain the problem branches create, then how the chip solves it, then what it means for profiling.

A modern CPU pipeline is roughly fifteen stages deep. From the moment an instruction enters the pipeline to the moment its result is committed is fifteen cycles. The CPU achieves throughput close to one instruction per cycle by keeping the pipeline always full: while instruction *i* is in stage 15, instruction *i+1* is in stage 14, and so on down to a fresh instruction entering stage 1. This works perfectly for straight-line code. But what happens at a branch?

Say the code is *if (x > 0) goto label_A; else goto label_B;*. The branch decision depends on the value of x. The value of x might not yet be ready. The CPU does not want to stall fifteen cycles waiting. So instead, it *guesses*. It predicts which way the branch will go, speculatively fetches and decodes instructions along the predicted path, and runs them. If the guess was right, the work was real and the pipeline never stalled. If the guess was wrong, the CPU must discard all that speculative work, flush the pipeline, and start over. The cost of a wrong guess is roughly fifteen cycles. A mispredicted branch is, in raw cycle terms, comparable to an L2 cache miss.

The branch predictor itself is a small dedicated piece of hardware that maintains a history of past branch outcomes. It uses that history to predict the next outcome. For branches that almost always go the same way (a loop's back-edge that runs a million times before terminating), the predictor gets it right effectively a hundred percent of the time. For random branches (data-dependent decisions on truly unpredictable data), the predictor gets it right roughly fifty percent. The branch misprediction rate is one of the key hardware counters.

Branches in matmul are easy: the loop tests are highly predictable, the loops always iterate to completion. The branch misprediction rate for naive matmul is near zero. We will see this when we run perf stat. For Kraken-2, branches inside the hash lookup may be less predictable, but cache misses overwhelm them so much that branch prediction is a second-order concern.

> **Key Insight**
> Branch mispredictions and cache misses cost similar numbers of cycles, but cache misses are far more common in real programs. When perf shows you a high misprediction rate (say above two percent), you should care. When it shows you a high L3 miss rate (say above ten percent), you should be alarmed.

2.3 Out-of-Order Execution and Why Instruction Count Lies

Here is one of the most counter-intuitive parts of modern CPUs. The order in which instructions are written in your binary is not the order in which they are executed. Modern x86 cores are **out-of-order** execution engines. They look at a window of, say, two hundred upcoming instructions, find ones whose inputs are ready, and execute them in whatever order makes the most use of the available execution units.

Each CPU core has several execution units. A typical modern core has multiple integer ALUs, two FP units, two load units, one store unit. If your code has independent operations, multiple can run in the same cycle. The peak rate is often around four micro-operations per cycle. The actual achieved rate is called **IPC**, instructions per cycle.

IPC is the single most important derived number that perf stat will give you. The formula is dead simple:

```math
IPC = instructions / cycles
```

If your IPC is high, the CPU is making good use of its execution units. If your IPC is low, the CPU is stalling. The question is always *why* is it stalling. The two big answers are cache misses and branch mispredictions, which is why those are the next counters perf will show you.

What is a good IPC? It depends on the workload. For compute-heavy code with good cache behavior, IPC of 2 to 4 is achievable on modern hardware. For memory-bound code, IPC of 0.3 to 0.7 is typical. For pointer-chasing code (linked lists, hash tables) IPC can drop below 0.2. Our naive matmul will have IPC near 0.3. Our ikj matmul will jump to around 2.1. That single number tells the whole story of why one is 10x faster than the other.

There is a deeper, more subtle point hiding here. Because of out-of-order execution, the *instruction count* of your program does not tell you how long it runs. Two programs that execute the same number of instructions can have wildly different runtimes if one has good IPC and the other has bad IPC. This is why *perf stat* reports both instructions and cycles. The ratio is what matters.

> **Common Pitfall**
> Beginners look at assembly and count instructions to estimate performance. This was approximately true for in-order chips like early RISC, but it is completely false for modern out-of-order x86 and ARM. Two assembly listings of the same length can run at radically different speeds depending on how well their data dependencies and memory access patterns play with the chip. Always measure cycles, never trust instruction count.

2.4 The IPC Compass

I want to give you a quick mental compass for reading IPC numbers. When perf stat hands you an IPC value, here is roughly what it means.

- **IPC > 3.0:** excellent. The CPU is genuinely doing work most cycles. Usually only possible with vectorized arithmetic and great cache behavior. Aim for this on compute-heavy kernels.
- **IPC 2.0 to 3.0:** good. Normal for well-tuned compiled code. Most cycles are productive. Optimization here is squeezing the last twenty percent.
- **IPC 1.0 to 2.0:** mediocre. Probably some cache misses or branch overhead. Room to improve.
- **IPC 0.5 to 1.0:** poor. Significant memory stalling. Cache analysis is the next step.
- **IPC < 0.5:** bad. The CPU is stalled most cycles. Almost always memory bound. Look at L3 miss rate and DRAM bandwidth.

I treat IPC as the first number I look at on any profile. It is the temperature reading of the program. If IPC is high, you are compute-limited and optimization means reducing arithmetic. If IPC is low, you are memory-limited and optimization means reducing memory traffic.

2.5 The Worked Example: Naive Matmul, Cycle by Cycle

Now we put it all together. Let me walk through what actually happens when the innermost loop of *matmul_naive* executes a single iteration.

```c
for (int k = 0; k < n; k++)
    C[i*n + j] += A[i*n + k] * B[k*n + j];
```

Each iteration does: one load of *A[i*n + k]*, one load of *B[k*n + j]*, one load of *C[i*n + j]*, one multiply, one add, one store of *C[i*n + j]*. That is roughly six operations per iteration. The compiler at -O2 will fold many of these into FMA (fused multiply-add) instructions and will keep *C[i*n + j]* in a register across iterations, hoisting the load and store out of the inner loop. So in practice each iteration is closer to: one load A, one load B, one FMA. Three operations.

If everything hits in L1 cache, those three operations finish in about one cycle of effective throughput, because the load units and FMA unit can all run in parallel. The ideal performance is roughly 1 cycle per iteration. With N = 1024, the total compute is N cubed = 1.07 billion iterations. At 3 GHz, 1.07 billion cycles = 0.36 seconds. That is the ideal.

Now reality. The load of *A[i*n + k]*, as k increments, walks stride 1 through memory. After eight iterations it crosses a 64-byte cache line boundary and triggers one cache fetch. So A misses about once every eight iterations. The prefetcher easily keeps up. A is essentially free.

The load of *B[k*n + j]*, as k increments, walks stride n through memory. For N = 1024 doubles, that is 8192 bytes, or 128 cache lines per step. Every single iteration accesses a different cache line. The prefetcher cannot help. Every iteration must wait for the cache line to arrive.

Where does the line come from? On the very first j-iteration of a given i, B is cold. Every k access misses all three caches and goes to DRAM. That is 200 cycles for each of N = 1024 iterations. After the first j-pass, the entirety of B has been touched, but B is 8 MB total (1024 * 1024 * 8 bytes). The L3 is around 8 to 32 MB. So B may or may not fit in L3 depending on the chip. On a chip with 8 MB L3, B exactly fills it, evicting everything else including A and C. On every subsequent j-iteration, the B access pattern still strides through cache lines, but they may be in L3 instead of DRAM. So instead of 200 cycles per miss, it is 40 cycles per miss.

Let us be generous and assume an 8 MB L3 holds all of B after the first pass. Then per inner iteration we pay roughly 40 cycles for the B access. Compared to the ideal of 1 cycle per iteration, we are running at 1/40 of peak. IPC will be near 0.1. The CPU is sitting idle 39 out of every 40 cycles. That is exactly the kind of pathological number perf stat will reveal.

Now fix it. The *matmul_ikj* version reorders to *i, k, j*, hoists *A[i*n + k]* out of the inner loop, and accesses B with stride 1 in the inner loop. Now both A and B are stride-1. Both have spatial locality. The prefetcher loves us. Almost every inner iteration is an L1 hit. We approach the ideal one cycle per iteration. IPC jumps from 0.1 to 2.1. The whole program speeds up by perhaps 10x without changing the algorithm at all. We changed three letters.

And Here Is Where It Gets Beautiful

The compiler did not save us. -O2 is fine, but it will not reorder a loop nest like that on its own, because it cannot prove it is safe in general. The hardware did not save us. The CPU was perfectly capable of running fast all along. What saved us was understanding that the memory hierarchy is the real performance story, and writing the loop to play with it instead of against it. That is the entire point of this chapter and the entire reason profiling exists. You measure, you understand the why, you change the code to match the hardware, and the chip rewards you tenfold.

2.6 What the PMU Will Be Counting

By the end of Chapter 3 you will be running perf stat and looking at numbers. Let me preview which numbers come from which parts of this chapter so the connection is immediate.

- **cycles:** total CPU cycles consumed. Driven by IPC and the size of the work.
- **instructions:** total instructions retired. Almost the same for naive and ikj matmul. The difference between them is not the instructions, it is the cycles.
- **cache-references:** number of L3 cache lookups. High for memory-bound workloads.
- **cache-misses:** number of L3 cache misses. Each miss costs a DRAM round-trip.
- **L1-dcache-load-misses:** L1 data cache misses. The first sign of poor spatial locality.
- **branches and branch-misses:** the branch predictor's report card. Almost irrelevant for matmul, sometimes critical for control-flow-heavy code.
- **page-faults:** the OS had to map a new virtual memory page. Major page faults touch disk and cost milliseconds.
- **context-switches:** the OS preempted your thread. Each one costs cache state.

Every one of these numbers is a window into a phenomenon we just discussed. cycles measures total time. instructions divided by cycles is IPC. cache-misses tells you how often the hierarchy failed you. branch-misses tells you how often the predictor failed you. By the end of the next chapter you will be reading these together as a coherent diagnostic, not as isolated numbers.

How This Connects to Your Project

For Dorado, none of these CPU counters matter directly because the work is on the GPU. We will care about the GPU equivalents: SM activity, DRAM throughput, L2 hit rate, warp stall reasons. The underlying philosophy is identical. The hierarchy on the GPU has registers, shared memory, L2, and HBM. The same locality logic applies.

For Kraken-2, every counter above will be screaming at us. IPC will be in the basement (around 0.3). L3 miss rate will be in the seventies of percent. The hot function will be the hash lookup. We will see every concept from this chapter in living color on a real research workload. That is the point. The same physics that makes our toy matmul slow makes Kraken-2 slow. The fix is the same in shape: change the access pattern, restore locality, hand the prefetcher something to predict.

What We Just Learned

The memory hierarchy is the dominant performance story on modern hardware. CPUs are about twenty-five times faster at arithmetic than at fetching from DRAM. Caches close that gap, but only when programs have spatial and temporal locality.

Caches transfer in 64-byte lines. Stride-1 access touches eight doubles per line. Stride-n access touches one. The prefetcher amplifies stride-1 access for free and is helpless against random patterns.

Branch prediction lets pipelines stay full at branches. A misprediction costs about fifteen cycles, roughly an L2 access. Branch mispredictions matter, but cache misses matter far more.

Out-of-order execution reorders instructions for parallelism. Instruction count does not predict runtime. IPC does. Look at IPC first on any profile.

IPC of 2 or better means compute-bound. IPC of 0.5 or worse means memory-bound. The boundary is roughly where you stop optimizing arithmetic and start optimizing memory access patterns.

Naive matmul fails the prefetcher because of stride-n access to B. The ikj reordering restores stride-1 access and yields a 10x speedup. We can predict this without running the profiler. The profiler is for confirming and quantifying.

Before You Move On

Next chapter we run perf stat for the first time. Make sure you can answer in your sleep: what is the formula for IPC, what is a cache line, why does stride-1 access matter, what is the cost ratio between L1 and DRAM. If any of those is shaky, reread the relevant section. Chapter 3 will assume all of this as background and will move quickly through the perf interface.

Also try to estimate, before you read Chapter 3, what perf stat will print when run on matmul_naive. What will IPC be? What will cache-miss rate be? Write your guesses down. Then we will measure and see how close you got. Calibrating your guesses against reality is one of the most valuable habits a profiling engineer can build.

CHAPTER 3

perf stat: Reading Hardware Counters

Now we run our first real tool. Chapter 2 told you what is happening inside the CPU. Chapter 3 hands you the meter that reads it. The tool is *perf stat*, and it is the single most useful command in the entire Linux profiling ecosystem. If you only ever learn one profiler in your life, learn this one.

I want to motivate this carefully before we type a command. Your instinct, when a program feels slow, is to run *time* and look at the wall clock. That tells you almost nothing. It tells you the total seconds. It does not tell you whether those seconds were spent computing, waiting for memory, waiting for the disk, or waiting for the OS to come back from another task. The total time is a symptom. The hardware counters are the diagnosis. perf stat is what gives you the diagnosis in two seconds, with no source-code changes, no recompilation, and essentially zero overhead.

> **What perf stat Does in One Sentence**
> It runs your program under a kernel-level harness that programs the CPU's Performance Monitoring Unit to count selected events, lets your program run to completion at full speed, and prints the counter totals at the end.

3.1 What perf Is and Where It Comes From

The perf command is the userspace front end to a kernel subsystem called *perf_events*. The kernel side was merged into Linux around 2009, replacing an older patchwork of profilers. The userspace command is part of the Linux source tree, in *tools/perf*. On most distributions it ships as a separate package because it is tied to a specific kernel version. On Ubuntu it is in the *linux-tools-common* and *linux-tools-$(uname -r)* packages. On Fedora it is just *perf*.

Installation is straightforward on bare-metal Linux:

```bash
# Ubuntu/Debian
sudo apt install linux-tools-common linux-tools-$(uname -r)

# Fedora/RHEL
sudo dnf install perf

# Arch
sudo pacman -S perf

# verify
perf --version
```

On WSL2 things are more annoying. The default WSL2 kernel does not ship with the perf binary, and you cannot just install the Ubuntu package because the kernel headers and userspace tools

must match the WSL2 kernel version, not the Ubuntu version. You have two options. Option A is to use Microsoft's WSLg kernel build, which has a perf binary buried in the kernel source tree. Option B, which is what I always do, is to clone the kernel matching your *uname -r* output and build perf from source.

```bash
# inside WSL2
uname -r        # note the exact version, e.g. 5.15.146.1-microsoft-standard-WSL2

sudo apt install build-essential flex bison libelf-dev libdw-dev \
                 libssl-dev libnuma-dev libunwind-dev libslang2-dev \
                 libperl-dev python3-dev libiberty-dev libzstd-dev

git clone --depth=1 \
  https://github.com/microsoft/WSL2-Linux-Kernel.git wsl-kernel
cd wsl-kernel/tools/perf
make -j$(nproc)

sudo cp perf /usr/local/bin/
perf --version
```

> **WSL2 Reality Check**
>
> Even after you have perf working on WSL2, two counters will show <*not supported*>: *LLC-loads* and *LLC-load-misses*. Hyper-V virtualizes the LLC counters in a way that perf cannot read. Everything else works. You will get IPC, L1 misses, branch stats, page faults, and so on. For the LLC numbers specifically, we will use Cachegrind in Chapter 7. This workaround is non-negotiable on WSL2.

One more piece of setup. By default, the Linux kernel restricts who can read hardware counters. This is controlled by */proc/sys/kernel/perf_event_paranoid*. The default value on most distros is 2, which means even reading basic counters requires CAP_PERFMON or root. The friendly setting for development is -1 (anyone can do anything) or 0 (anyone can read most counters).

```bash
# temporarily allow unprivileged perf
sudo sysctl -w kernel.perf_event_paranoid=-1

# permanently
echo 'kernel.perf_event_paranoid=-1' | sudo tee /etc/sysctl.d/99-perf.conf
```

Without that, you will get errors like "You may not have permission to collect stats" and you will be confused for an hour. Set it once and forget it.

3.2 The Default Event Set

Running *perf stat* with no arguments selects a default set of events. The set varies slightly across kernel versions but is essentially these:

- **task-clock** — CPU time consumed by your task, in milliseconds. This is the sum of time across all your threads. For a single-threaded program it is close to wall-clock time. For an

8-threaded program it is roughly 8x wall-clock time.

- **context-switches** — how many times the OS preempted your task to run something else. Should be near zero for a tight CPU-bound run.
- **cpu-migrations** — how many times the OS moved your thread to a different physical core. Each migration costs cache state because the new core's caches are cold.
- **page-faults** — minor and major. Minor faults are first touches of mapped pages. Major faults touch disk. Major faults are catastrophic for performance.
- **cycles** — total CPU cycles consumed across all cores running your task. The denominator of IPC.
- **instructions** — total retired instructions. The numerator of IPC.
- **branches** — total branch instructions executed.
- **branch-misses** — total mispredicted branches.
- **cache-references** — L3 cache lookups (architecture-dependent; on most Intel chips this is LLC accesses).
- **cache-misses** — L3 cache misses.

Those ten counters are enough to diagnose perhaps eighty percent of all performance problems. The other twenty percent needs more specialized events, which we will get to in Section 3.5.

3.3 The Three Derived Metrics You Always Compute

perf stat will print the raw counters. The raw numbers are not the goal. The goal is three derived ratios, which I compute every single time without exception. Memorize these three:

IPC: instructions / cycles

We covered this in Chapter 2. IPC tells you how efficiently the CPU's execution units were used. perf stat prints this automatically as *insn per cycle* at the end of the row for instructions. Always look at it first. Target: above 2.0 for compute code. Anything below 1.0 means significant stalling.

Branch miss rate: branch-misses / branches

Tells you how often the branch predictor was wrong. Target: under one percent. Above five percent and your control flow is genuinely unpredictable, often by data-dependent branches on random inputs.

Cache miss rate: cache-misses / cache-references

Specifically, this is the L3 (LLC) miss rate. Target: under ten percent. Above thirty percent and you are bleeding cycles to DRAM. Above seventy percent and your workload is essentially streaming from memory, which is the pattern we expect for Kraken-2's hash lookup.

> **Key Insight**
> Two numbers diagnose ninety percent of slow programs. IPC tells you *how badly*. Cache miss rate tells you *why*. Low IPC plus high cache miss rate equals memory-bound. Low IPC plus low cache miss rate equals branch-bound or sync-bound. High IPC plus high anything equals you are doing well.

3.4 Running perf stat on Our Benchmarks

Now we measure. We compile the four CPU benchmarks at -O2, which is what a serious project would use. We use -O2 rather than -O3 deliberately, because -O3 turns on aggressive vectorization that can confuse the analysis. -O2 gives us optimized but understandable code.

```bash
gcc -O2 -o matmul_naive matmul_naive.c
gcc -O2 -o matmul_ikj   matmul_ikj.c
gcc -O2 -o matmul_tiled matmul_tiled.c
gcc -O2 -fopenmp -o matmul_omp matmul_omp.c
```

Now run perf stat on the naive version:

```bash
perf stat ./matmul_naive
```

On my Intel i7-10750H, the output looks like this. Numbers will vary on your machine but the shape is the same.

```output
 Performance counter stats for './matmul_naive':

          7,842.31 msec task-clock                #    1.000 CPUs utilized
                23      context-switches          #    2.933 /sec
                 0      cpu-migrations            #    0.000 /sec
             3,082      page-faults               #  392.989 /sec
    23,478,901,234      cycles                    #    2.994 GHz
     7,512,098,765      instructions              #    0.32  insn per cycle
       847,019,283      branches                  #  108.005 M/sec
         3,201,847      branch-misses             #    0.38% of all branches
     1,489,712,608      cache-references          #  189.957 M/sec
       687,432,109      cache-misses              #   46.14% of all cache refs

       7.844391282 seconds time elapsed
       7.836102000 seconds user
       0.005994000 seconds sys
```

Read this with me. Line by line.

task-clock 7842 msec. The program took 7.84 seconds of CPU time. For a single-threaded program this is basically wall time.

context-switches 23. Almost nothing. Good. The OS mostly left our program alone.

page-faults 3082. These are minor faults from first-touching the 24 MB of matrix memory. Each fault maps a 4 KB page, so 6144 pages worth of matrix memory, which matches three 1024x1024 double arrays of 8 MB each. Sanity check passed.

cycles 23.5 billion. *instructions 7.5 billion*. Now the punchline: *0.32 insn per cycle*. IPC of 0.32. The CPU executed less than a third of an instruction per cycle on average. By Chapter 2's compass, this is bad. The CPU is stalling about two cycles out of every three. Why?

cache-misses 687M / cache-references 1490M = 46% LLC miss rate. Forty-six percent. Half of all L3 lookups missed and went to DRAM. That is the why. The naive matmul is memory-bound, exactly as Chapter 2 predicted.

Now the same command on the ikj version:

```bash
perf stat ./matmul_ikj
```

```output
 Performance counter stats for './matmul_ikj':

            751.42 msec task-clock                #    0.999 CPUs utilized
                 2      context-switches          #    2.662 /sec
                 0      cpu-migrations            #    0.000 /sec
             3,084      page-faults               #    4.105 K/sec
     2,251,043,876      cycles                    #    2.996 GHz
     4,729,873,210      instructions              #    2.10  insn per cycle
       782,109,876      branches                  #  1.041 G/sec
         3,011,234      branch-misses             #    0.39% of all branches
        18,432,901      cache-references          #   24.530 M/sec
         1,043,287      cache-misses              #    5.66% of all cache refs

       0.752108293 seconds time elapsed
       0.749872000 seconds user
       0.002016000 seconds sys
```

Look at what happened. The same program, mathematically. Different loop order. *task-clock* dropped from 7842 ms to 751 ms. That is a 10.4x speedup. Why?

IPC jumped from 0.32 to 2.10. The CPU is now doing real work on most cycles. *cache-misses dropped from 46.14% to 5.66%*. The cache miss rate fell by a factor of eight. *cache-references* total dropped from 1.49 billion to 18 million, a factor of 80, because most accesses now hit in L1 and never even reach the L3 lookup.

That single comparison is everything. Two columns of the perf stat output (IPC and miss rate) tell you what changed and why. You do not need to read assembly. You do not need to instrument anything. You ran two commands and you have a complete diagnosis of a 10x performance difference. This is the power of hardware counters.

> **And Here Is Where It Gets Beautiful**
> We predicted this in Chapter 2 without running anything. Stride-n access to B in naive matmul should kill the prefetcher and the cache. Stride-1 access in ikj matmul should restore them. The PMU just confirmed our prediction with hard numbers. *That* is what good profiling feels like. You form a hypothesis from the architecture, you run the tool, the tool confirms the hypothesis. You do not discover the answer from the tool. You verify it.

Let me also show tiled and openmp briefly. Tiled blocks the loops to keep working sets in L1, so we expect IPC similar to ikj but with fewer total cache references.

```bash
perf stat ./matmul_tiled

# expected: task-clock around 680 ms, IPC around 2.4, cache-misses < 3%
```

Tiled is typically slightly faster than ikj because it improves temporal locality further. The naked eye difference is small on a 1024 matrix. On a larger matrix the gap widens.

OpenMP is interesting because perf stat aggregates counters across threads. With 8 threads, task-clock will be roughly 8x wall time, and instructions roughly the same total. Cycles per thread should be similar, so IPC stays high. Wall time should drop by close to 8x for a well-parallelized memory-friendly workload.

```bash
OMP_NUM_THREADS=8 perf stat ./matmul_omp
```

> **Common Pitfall**
> When you run perf stat on a multi-threaded program, the *task-clock* is the sum of CPU time across all threads, not wall time. To see wall time, look at the *seconds time elapsed* line at the bottom. Beginners report task-clock as elapsed time and report a 1x speedup on 8 threads, when in fact the program ran 7x faster.

3.5 Custom Events

The default set is good but limited. perf supports hundreds of events. You select them with *-e*. Some events that I use constantly:

```bash
perf stat -e L1-dcache-loads,L1-dcache-load-misses,\
L2-loads,L2-load-misses,LLC-loads,LLC-load-misses ./matmul_naive
```

This breaks down the cache hierarchy by level. You can see exactly where data is being found. For naive matmul you will see most loads miss L1, many miss L2, and a substantial fraction miss LLC.

For ikj the L1 miss rate alone drops to under one percent.

To list all available events on your machine:

```bash
perf list                           # everything
perf list cache                     # filter by keyword
perf list hw                        # only hardware events
```

Some other events I reach for:

- **dTLB-load-misses** — Data Translation Lookaside Buffer misses. A TLB miss means a virtual-to-physical address translation missed the TLB and had to walk the page tables, costing perhaps 50 to 100 cycles. Workloads with huge working sets (Kraken-2's 180 GB database) will hit this hard.
- **stalled-cycles-frontend** — cycles where the front-end could not deliver instructions. Indicates issues like instruction cache misses or branch mispredictions.
- **stalled-cycles-backend** — cycles where the back-end could not retire instructions. Usually due to memory stalls. This is the cleanest single indicator of memory boundness.
- **cs (context-switches), migrations** — useful for diagnosing OS interference.

You can combine these. The kernel will allocate them across the limited number of physical PMU counters and may multiplex if there are more events than counters. With multiplexing, the kernel rotates which events are active and scales the counts proportionally. The output will show a *(scaled)* tag if multiplexing happened.

```bash
# memory-bound diagnosis bundle that I use constantly
perf stat -e cycles,instructions,\
L1-dcache-loads,L1-dcache-load-misses,\
LLC-loads,LLC-load-misses,\
dTLB-loads,dTLB-load-misses,\
stalled-cycles-backend ./matmul_naive
```

3.6 Running perf stat on Kraken-2

Now the connection to the real project. Kraken-2 classifies sequencing reads by matching k-mers against a hash table. The hash table for the ESKAPE pathogen database is around 180 GB on disk and mostly memory-mapped at runtime. Let us measure what running it looks like under perf stat.

```bash
# basic perf stat on a small Kraken-2 run
perf stat kraken2 --db /path/to/eskape_db \
                  --threads 8 \
                  --output classifications.tsv \
                  sample_reads.fastq
```

What I expect to see, and what you should expect to see when you run this on the sauron machine or the LUNA cluster:

- **IPC** in the range 0.4 to 0.8. Far below the matmul ikj value of 2.1. This is because hash lookups are dependency chains of pointer dereferences that the out-of-order engine cannot reorder around.
- **cache-miss rate** above 50%, likely in the 70-80% range. Each k-mer lookup is a random access into a multi-GB table. The L3 will almost never contain the line you want.
- **dTLB-load-misses** elevated, because the working set is far larger than the TLB can cover. A 180 GB database with 4 KB pages needs 45 million TLB entries; the TLB has perhaps 1500 entries. Almost every random access incurs a TLB miss.
- **page-faults** may be significant on the first run, because mmap'd pages are only resident after first touch. A second run with the database already paged in will be much cleaner.
- **branch-misses** moderate, maybe one to two percent. Hash table probing has some unpredictable branches but they are not the bottleneck.

> **How This Connects to Your Project**
>
> The two-page report needs exact numbers for Kraken-2's cache miss rate. perf stat is the tool that gives you them. Run it, capture the output, copy the IPC and the cache-miss percentage straight into the report. Numbers like "IPC 0.6, LLC miss rate 78%" are the difference between a report that says "Kraken-2 is slow" and a report that says "Kraken-2 spends 78% of L3 lookups in DRAM at IPC 0.6, indicating a memory-bandwidth bottleneck on hash table accesses." The second sentence is what Prof. Paul wants.
>
> If you are running on WSL2 the LLC numbers will say *<not supported>*. Use Cachegrind instead (Chapter 7) and put the simulated number in the report with a footnote. On sauron and LUNA you will get the real number from perf stat directly.

3.7 A Few Idioms That Will Save You

Three little flags that change perf stat from useful to indispensable.

Multiple runs and statistics: -r

A single run is noisy. perf stat can repeat the run and show statistics:

```bash
perf stat -r 5 ./matmul_naive

# at the end:
#   7.840 +- 0.012 seconds time elapsed  ( +-  0.15% )
```

The +- value is the standard deviation. If your runs vary by more than a few percent, something else on the machine is interfering. Always report median or mean of multiple runs in your write-ups.

Attach to a running process: -p

If your program is already running and you want to start counting from now:

```bash
# in one terminal:
./long_running_program &
pid=$!

# in another terminal:
perf stat -p $pid sleep 10    # count for 10 seconds, then print
```

This is the right way to profile a server, a daemon, or a long-running pipeline like Dorado where you do not want to include startup time in the measurement.

Per-CPU breakdown: -A

For multi-threaded programs, -A shows per-CPU totals:

```bash
OMP_NUM_THREADS=8 perf stat -A ./matmul_omp
```

Each CPU gets its own row. You can see if work was balanced across cores or if one core was doing more. Imbalance is a sign of poor scheduling, false sharing, or memory bandwidth contention.

3.8 Reading the Output Like a Doctor

Let me give you the diagnostic flow I use every time. When I get a perf stat output, I read it in this order:

- **1. Wall time.** Bottom line, *seconds time elapsed*. Is the program slow or fast in absolute terms?
- **2. IPC.** The most informative single number. Compute-bound (IPC > 2) or memory-bound (IPC < 1)?
- **3. Cache miss rate.** If IPC is low, this is the most likely culprit. Above thirty percent is a red flag.
- **4. Branch miss rate.** Almost always low. If it is above one percent, look at unpredictable branches.
- **5. Page faults.** Above a few hundred per second indicates the OS is doing a lot of work paging memory in. May suggest swap pressure or first-touch effects.
- **6. Context switches and migrations.** Should be near zero for a clean run. If high, something is interfering and you should rerun with the system quieter.

Most of the time, IPC and cache miss rate together give you the answer. The other counters are corroboration. With practice this whole diagnosis takes ten seconds per program.

Common Pitfall

Compiling with *-O0* for debug and then profiling. The optimized binary is a different program with different cache behavior. Always profile -O2 or -O3 builds.

Forgetting *-g* when you plan to do source annotation later. We will need this in Chapter 4. Always compile with *-O2 -g* for profilable builds. -g adds debug info without changing optimization.

Running perf stat on a program that finishes in milliseconds. The counters' statistical accuracy is poor for very short runs. Either give the program more work or use repetition with -r.

What We Just Learned

perf stat runs your program at full speed and prints hardware counter totals at the end. The overhead is essentially zero.

Three derived ratios are the key diagnostic outputs: IPC (instructions per cycle), branch miss rate, and cache (LLC) miss rate. Compute them every time.

Naive matmul has IPC 0.32 and LLC miss rate 46%. Ikj matmul has IPC 2.10 and LLC miss rate 5.66%. The 10x speedup is fully explained by these two numbers.

WSL2 cannot read LLC counters. Use Cachegrind for those. All other counters work on WSL2.

Always compile with -O2 -g. Always use -r for repeatable runs. For long-running programs attach with -p instead of starting them under perf.

On Kraken-2 we expect low IPC (0.4 to 0.8) and very high cache miss rate (50-80%), consistent with random-access pointer chasing in a multi-GB hash table.

Before You Move On

perf stat tells you *what* happened. It does not tell you *where*. To find which function is the hotspot, you need perf record and perf report. That is Chapter 4.

Before reading Chapter 4, run perf stat on a piece of your own code. Anything. Pick a function you suspect is slow. Run perf stat -r 5 on it. Compute IPC and cache miss rate. Form an opinion. This is how you build the habit of measuring before guessing.

CHAPTER 4

perf record: Finding Hotspots

perf stat told us *what* happened: the program had a 46% cache miss rate and an IPC of 0.32. It did not tell us *where*. Which function? Which line of source? Which assembly instruction? For our two-line matmul it is obvious, because there is only one loop. For Kraken-2 or Dorado, which are tens of thousands of lines, perf stat alone is useless for localizing the problem. We need a tool that builds a map of *where in the code* time was spent. That tool is perf record, and its companion perf report.

Let me motivate the mechanism, because it is genuinely clever. perf stat counts events globally. It cannot attribute a cache miss to a line of code, because the counter does not know what line was executing. perf record solves this with sampling. Many thousands of times per second, it interrupts the CPU and records the current instruction pointer and call stack. After the run, it has hundreds of thousands of samples. It buckets them by function and by address. A function that received forty percent of samples consumed roughly forty percent of CPU time. The map of where samples landed is the map of where time went.

> **The Core Idea in One Sentence**
> perf record interrupts your program at a fixed frequency, records where it was each time, and builds a histogram of program location, which is a statistical map of where time was spent.

4.1 Basic Usage

The simplest invocation. Record, then report.

```bash
gcc -O2 -g -o matmul_naive matmul_naive.c   # note the -g

perf record -g ./matmul_naive
perf report
```

Two flags matter here. The *-g* on the compile line embeds debug information so perf can map addresses back to source lines and function names. The *-g* on the perf record line is a different -g entirely; it tells perf to capture the full call stack at each sample, not just the leaf instruction pointer. Same letter, two completely different meanings, on two different commands. This trips everyone up once.

perf record writes a file called *perf.data* in the current directory. perf report reads it back and opens an interactive TUI. For matmul_naive the report is almost comically simple:

output

```
# Overhead  Command       Shared Object      Symbol
# ........  ............  .................  ............................
#
    98.71%  matmul_naive  matmul_naive       [.] matmul_naive
     0.84%  matmul_naive  [kernel.kallsyms]  [k] clear_page_erms
     0.21%  matmul_naive  libc.so.6          [.] __memset_avx2_unaligned
     0.14%  matmul_naive  matmul_naive       [.] main
     0.10%  matmul_naive  [kernel.kallsyms]  [k] asm_exc_page_fault
```

Read this top-down. The *Overhead* column is the percentage of samples. *matmul_naive* (the function, not the binary, shown in the Symbol column with a *[.]* meaning userspace) received 98.71% of all samples. That is your hotspot. Everything else is noise: page clearing by the kernel, the calloc-driven memset, main's setup. The story is unambiguous: 99% of time is in the matmul function. No surprise here, but on a large program this is exactly how you find the needle.

The interactive TUI lets you press Enter on a symbol to drill in, *a* to annotate, + to expand call stacks. Spend ten minutes navigating it on a real program and it becomes second nature.

4.2 perf annotate: Down to the Assembly

98.71% in the matmul function is a start, but inside that function, which *line* is hot? perf annotate maps samples onto source and assembly together.

bash

```
perf record -g ./matmul_naive
perf annotate matmul_naive
```

Or press *a* on the symbol inside perf report. The output interleaves your C source with the generated assembly and shows the sample percentage on each instruction. For naive matmul, almost all the samples land on a single instruction in the inner loop:

output

```
         :      for (int k = 0; k < n; k++)
         :          C[i*n + j] += A[i*n + k] * B[k*n + j];
    2.41 :   movsd  (%rdx),%xmm0          # load A[i*n+k]
   91.18 :   mulsd  (%rcx,%rax,8),%xmm0   # load B[k*n+j], multiply  <-- HOT
    3.02 :   addsd  %xmm1,%xmm0           # accumulate into C
    1.88 :   movsd  %xmm0,(%rsi)          # store C[i*n+j]
    1.51 :   add    $0x1,%r8d             # k++
```

Ninety-one percent of all samples are sitting on the *mulsd* instruction. But here is the subtle and important reading. That instruction is not slow because multiplication is slow. Multiplication is one cycle. It is slow because the *mulsd* instruction has an embedded memory operand: *(%rcx,%rax,8)* is the load of *B[k*n + j]*. The instruction stalls because the memory load misses cache. The samples pile up on the instruction that is waiting for memory.

> **A Crucial Subtlety**
> Sampling attributes a stall to the instruction the CPU was waiting on. When you see ninety percent of samples on one memory-touching instruction, the lesson is almost never "that instruction is computationally expensive." It is "that instruction is waiting for memory." Reading annotate output correctly means recognizing that a hot instruction with a memory operand is a cache miss in disguise. This is precisely the B column-access miss we predicted in Chapter 2 and confirmed with perf stat in Chapter 3, now localized to a single assembly instruction.

Run the same annotate on matmul_ikj and the samples spread out across the inner loop instead of piling on one. The memory operand for B is now stride-1 and hits L1, so the instruction no longer stalls. The visual difference between the two annotate outputs is the entire optimization story in one screen.

4.3 Flame Graphs: The Most Readable Visualization

perf report's TUI is functional but ugly, and for deeply nested call stacks it is hard to see the big picture. The single best visualization of profiling data ever invented is the flame graph, created by Brendan Gregg. It turns a forest of sampled call stacks into a single immediately-legible image.

Here is how to make one. You need the FlameGraph scripts, which are a small set of Perl scripts on GitHub.

```bash
git clone https://github.com/brendangregg/FlameGraph.git

perf record -F 99 -g ./matmul_naive
perf script | ./FlameGraph/stackcollapse-perf.pl \
            | ./FlameGraph/flamegraph.pl > matmul_naive.svg

# open matmul_naive.svg in any browser
```

Let me explain that pipeline. *perf record -F 99 -g* samples at 99 Hz with call stacks. *perf script* dumps the raw samples as text. *stackcollapse-perf.pl* folds each unique call stack into a single line with a count. *flamegraph.pl* renders those folded stacks as an interactive SVG.

How to read a flame graph

This is worth slowing down for, because the layout is unintuitive at first and obvious once it clicks.

- **The x-axis is NOT time.** It is alphabetically sorted samples. Do not read left-to-right as a timeline. This is the single most common misreading.
- **Width is what matters.** The wider a box, the more samples landed in that function (and its children). Wide means hot.
- **The y-axis is stack depth.** The bottom box is main. Each box above it is a function called by the box below. The flames go up as the call stack goes deeper.
- **The top edge is where the CPU actually was.** The topmost box at any x-position is the leaf function executing at that sample. Everything below it is the call chain that led there.

- **You look for wide plateaus near the top.** A wide box at the top means the CPU spent a lot of time directly in that function. That is your hotspot.

For matmul_naive the flame graph is boring: one tall narrow stack (main calls matmul_naive) with one enormously wide plateau (matmul_naive) taking up 99% of the width. Boring is correct here. The value of flame graphs explodes on complex software where the call tree is deep and branchy, which is exactly what Kraken-2 and Dorado are.

The -F flag and overhead

The *-F* flag sets the sampling frequency in Hz. The trade-off is accuracy versus overhead.

- **-F 99** samples 99 times per second. Overhead around one percent. The standard choice. (99 rather than 100 to avoid lockstep with timers that fire at round frequencies, which would bias the samples.)
- **-F 999** samples roughly a thousand times per second. Overhead a few percent. Use for short programs needing more samples.
- **-F 9999** samples ten thousand times per second. Overhead around ten percent. Use only for very short hot regions where you need fine resolution.

I default to -F 99 for anything that runs more than a second or two. Higher rates only when the program is too short to accumulate enough samples otherwise.

4.4 Event-Based Sampling: Sample on Cache Misses

Here is one of the most powerful features and one almost nobody uses. By default perf record samples on a timer, which gives a where-is-time-spent map. But you can sample on *any* hardware event. If you sample on cache misses, you get a map of where cache misses happen, which is far more targeted.

```bash
perf record -e cache-misses -g ./matmul_naive
perf report
```

Now the histogram is weighted by cache misses, not by time. Each sample is taken after every N cache misses (the period auto-tunes). The functions and lines that dominate this report are the ones generating the most cache misses. For naive matmul it is, again, the B-access instruction in the inner loop. But the conceptual power is huge: on a complex program, sampling on cache-misses tells you precisely which code is thrashing the cache, even if that code is not the overall time hotspot.

You can do the same with any event. Sample on *branch-misses* to find unpredictable branches. Sample on *dTLB-load-misses* to find code touching too much memory. This is the targeted-diagnosis mode of perf record, and it is exactly what you want when perf stat has already told you the problem is, say, cache misses, and now you want to know where they come from.

> **Key Insight**
> perf stat tells you the problem is cache misses. perf record -e cache-misses tells you where the cache misses are. perf annotate tells you which instruction. That chain (stat to record to annotate) is the canonical workflow for a memory-bound program, and you will run it on Kraken-2 almost exactly as written.

4.5 Finding the Hotspot in Kraken-2

Now the real thing. We run the exact same workflow on Kraken-2 to find which function dominates classification time and which function generates the cache misses.

```bash
# step 1: where is time spent?
perf record -g -- kraken2 --db /path/to/eskape_db \
                          --threads 8 \
                          sample_reads.fastq > out.tsv
perf report --stdio | head -30
```

The double dash before kraken2 separates perf's own options from the program's options, which matters because kraken2 has flags that perf would otherwise try to interpret. Expect a report where the dominant symbol is the k-mer hash lookup or the database query routine. The exact name depends on the Kraken-2 build, but it will be a function with *hash*, *lookup*, *query*, or *compact* in the name.

```bash
# step 2: where are the cache misses?
perf record -e cache-misses -g -- kraken2 --db /path/to/eskape_db \
                                          --threads 8 \
                                          sample_reads.fastq > out.tsv
perf report --stdio | head -30
```

This second report is the money shot for the project. It will show the hash-lookup function dominating cache misses, confirming that the memory-bound behavior we saw in perf stat originates exactly where we hypothesized: in the random-access hash table query. That confirmation is a sentence in the report, backed by a concrete percentage.

> **How This Connects to Your Project**
>
> The two-page report's Kraken-2 page needs a "hotspot functions" subsection. perf record gives you exactly that, as a ranked list with percentages. Capture the top five symbols and their overhead percentages. Then run with -e cache-misses to show that the hotspot function is also the cache-miss generator. Two reports, one clean narrative: time is concentrated in the hash lookup, and that hash lookup is dominated by cache misses, therefore the optimization target is the cache behavior of the hash lookup.
>
> This is also the exact evidence that justifies the Hot-K-mer LRU cache. The cache-miss-weighted perf report is the data that says "this specific function thrashes memory." When you later add the cache and rerun, the same report should show the function's cache-miss share dropping. That before-and-after is the strongest possible argument that your optimization worked.

One practical note for Dorado. perf record will profile the CPU side of Dorado, which is mostly orchestration: reading pod5 files, queuing work, copying buffers. The actual neural network runs on the GPU and is invisible to perf. So perf record on Dorado tells you about the host overhead, which is useful for finding data-loading bottlenecks, but the kernel work needs Nsight. We will get there in Batch 3. For now, know that perf is the wrong tool for the GPU half of the pipeline.

4.6 Recording with Specific Options

A few perf record options worth knowing.

- **--call-graph dwarf** instead of *-g*: uses DWARF debug info to unwind stacks instead of frame pointers. More accurate when code is compiled without frame pointers (common at -O2), but larger perf.data and slower processing. Use it when -g gives broken or truncated stacks.
- **-o filename**: write to a named file instead of perf.data. Essential when comparing runs (Chapter 5's perf diff needs two named files).
- **-p PID**: attach to a running process, same as perf stat. Profile a server without restarting it.
- **-a**: system-wide. Sample every CPU, every process. Useful for finding which process on a busy machine is hogging the CPU.
- **--**: separates perf options from the target command's options. Use it whenever the target has its own flags.

```bash
# accurate stacks for an -O2 build without frame pointers
perf record --call-graph dwarf -F 99 -- ./matmul_omp

# named output for later comparison
perf record -o naive.data ./matmul_naive
perf record -o ikj.data   ./matmul_ikj
```

4.7 The Sampling Caveat You Must Internalize

Sampling is statistical. That has two consequences you must respect.

First, short programs give too few samples. At 99 Hz, a program that runs for 50 milliseconds produces about five samples. Five samples cannot localize a hotspot. Either make the program do more work, raise the frequency with -F, or switch to an instrumentation tool like Callgrind (Chapter 8) that counts exactly.

Second, sampling has skid. When the PMU fires an interrupt on a cache-miss event, the instruction pointer recorded is not exactly the instruction that caused the miss. It is a few instructions later, because of pipeline depth and interrupt latency. This is called skid. It means perf annotate can attribute a miss to the instruction just after the real culprit. Modern Intel chips offer PEBS (Precise Event-Based Sampling) which dramatically reduces skid; you enable it with the *:pp* or *:ppp* suffix on an event.

```bash
# precise sampling to reduce skid
perf record -e cache-misses:pp -g ./matmul_naive
```

Use the precise modifier whenever you are doing instruction-level annotation of a specific event. For coarse function-level hotspot finding, skid does not matter. For line-level blame, it does.

Common Pitfall

Reading the flame graph x-axis as time. It is not time. It is sorted samples. Width is the only meaningful horizontal quantity.

Forgetting -g on the compile line, which leaves perf unable to resolve function names, so you get a report full of raw hex addresses.

Profiling a program too short to sample. Five samples tell you nothing. Give it work or raise the frequency.

Trusting instruction-level annotate without the precise (:pp) modifier. Skid will lie to you about which exact instruction caused a memory event.

What We Just Learned

perf record samples the program location thousands of times per second and builds a histogram. perf report shows the ranked hotspots by sample percentage.

perf annotate maps samples onto source and assembly. A hot instruction with a memory operand is usually a cache miss, not expensive arithmetic.

Flame graphs are the most readable visualization. Width is samples, height is stack depth, the top edge is where the CPU actually was. The x-axis is not time.

Event-based sampling (perf record -e cache-misses) builds a map weighted by cache misses instead of time, localizing exactly where a memory problem originates.

The canonical memory-bound workflow is perf stat to diagnose, perf record -e cache-misses to localize, perf annotate to pinpoint the instruction. You will run this on Kraken-2 nearly verbatim.

Sampling is statistical. Short programs starve it of samples. Use the :pp precise modifier for accurate instruction-level event attribution.

Before You Move On

You now have the core perf toolkit: stat for what, record and report for where, annotate for which instruction. Chapter 5 takes perf into advanced territory: perf diff for before-and-after comparison, perf mem for memory-access profiling, perf c2c for false sharing, and the crown jewel, the Roofline Model that answers how-much-faster.

Before continuing, generate a flame graph of any program you have, even a trivial one, and open the SVG in a browser. Hover over the boxes. Click to zoom. Build the muscle memory now, on something simple, so that when you flame-graph Kraken-2 the visualization is familiar and only the data is new.

CHAPTER 5

perf Advanced and the Roofline Model

We close out the perf chapters with the advanced toolkit and the single most important conceptual framework in all of performance engineering: the Roofline Model. Up to now perf has answered *what* and *where*. This chapter answers *what changed* (perf diff), gives finer memory detail (perf mem and perf c2c), and finally answers the question almost nobody asks but everybody should: *how much faster could this possibly go?* That last question is the roofline, and it will change how you think about optimization permanently.

5.1 perf diff: What Changed

You optimized something. Or a library upgrade slowed you down. Or you flipped a compiler flag. The question is: relative to before, where did time move? perf diff compares two perf.data files and shows the delta per function.

```bash
# record before and after with named outputs
perf record -o naive.data ./matmul_naive
perf record -o ikj.data   ./matmul_ikj

# compare
perf diff naive.data ikj.data
```

The output shows each symbol with its overhead in both runs and the difference. For our matmul comparison the matmul function still dominates both, but the absolute time collapsed. perf diff is most powerful in regression hunting: you record a known-good baseline, you record the suspected-slow build, and perf diff points straight at the function whose share grew. Instead of reading two reports side by side and doing arithmetic in your head, perf diff does the subtraction for you.

A realistic workflow for the project: record Kraken-2 before adding the Hot-K-mer cache, save it as *before.data*. Add the cache. Record again as *after.data*. perf diff before.data after.data shows you exactly which functions got cheaper and by how much. That single command produces the core evidence for your optimization write-up.

> **How This Connects to Your Project**
>
> perf diff is the tool that quantifies your optimizations for the report. "The Hot-K-mer cache reduced the lookup function's overhead from 61% to 24% of total samples" is a perf diff result, and it is exactly the kind of concrete before/after number that makes a profile report convincing rather than hand-wavy.

5.2 perf mem: Memory Access Profiling

perf record -e cache-misses tells you which code misses the cache. perf mem goes deeper: it tells you, for memory access samples, where the data came from (L1, L2, L3, local DRAM, remote DRAM on a NUMA system) and how long each access took. It uses the same PEBS precise-sampling machinery to attribute loads and stores accurately.

```bash
perf mem record ./matmul_naive
perf mem report
```

The report adds columns for the memory level that satisfied each access and the access latency. For naive matmul you will see a large fraction of loads satisfied by *Local RAM* or *LFB/L3 miss* with high latencies in the hundreds of cycles, confirming the DRAM-bound nature of the B access. For ikj you will see the same loads satisfied by *L1* at four-cycle latency. perf mem is the most direct way to see the memory hierarchy level distribution of your accesses, which is precisely the picture Chapter 2 drew abstractly.

On a NUMA machine (the LUNA cluster nodes, the sauron machine if it has multiple sockets), perf mem distinguishes local from remote DRAM. Remote DRAM accesses cross the inter-socket link and cost roughly double a local access. If perf mem shows many remote accesses, the fix is NUMA-aware allocation (numactl, which we cover in Chapter 17), pinning the data and the threads to the same socket.

5.3 perf c2c: Cache-to-Cache and False Sharing

This one is specialized but it saves multi-threaded programs from a subtle, vicious bug called false sharing. Let me motivate it, because the symptom is bizarre: you parallelize a loop across eight threads and it gets *slower* than single-threaded, and perf stat shows a high cache miss rate even though each thread touches different data.

False sharing happens when two threads write to different variables that happen to live on the same 64-byte cache line. The hardware maintains cache coherence at cache-line granularity. So when thread A on core 0 writes its variable, the entire line is invalidated in core 1's cache, even though thread B's variable on that line was untouched. Thread B then re-reads the line, A writes again, B's copy is invalidated again. The line ping-pongs between cores' caches through the coherence protocol. The threads are not sharing data, but they are sharing a cache line, hence "false" sharing. The performance collapse can be tenfold.

```bash
perf c2c record ./matmul_omp
perf c2c report
```

perf c2c identifies cache lines that bounce between cores and points at the exact lines and offsets involved. The fix is to pad data so each thread's working data sits on its own cache line, or to restructure so threads write to thread-local buffers and combine at the end. Our matmul_omp parallelizes over rows of C, and different rows are far apart in memory, so false sharing is unlikely.

But the moment you parallelize a reduction or a shared counter, perf c2c becomes essential.

> **Common Pitfall**
> When an OpenMP loop runs slower than serial, the reflex guess is "thread overhead" or "too few iterations." Often the real cause is false sharing on a shared cache line. perf c2c is the only tool that names it directly. Reach for it whenever parallel is mysteriously slower than serial.

5.4 The Roofline Model: How Fast Could This Possibly Go?

Now the centerpiece. Everything before this answered questions about what your program *is* doing. The roofline answers what your program *could* be doing. It is the framework that tells you whether to keep optimizing or to stop and go home. I consider it the single most important idea a performance engineer can hold in their head.

The model rests on one observation. Any computation is limited by one of two ceilings: how fast the machine can do arithmetic (peak compute, in FLOPs per second), or how fast it can move data (peak memory bandwidth, in bytes per second). Which ceiling binds you depends on a property of your algorithm called arithmetic intensity.

Arithmetic Intensity

Arithmetic intensity (AI) is the ratio of floating-point operations performed to bytes of DRAM traffic required.

```math
Arithmetic Intensity = FLOPs performed / bytes moved from DRAM
```

AI has units of FLOPs per byte. A low AI means you do little arithmetic per byte fetched, so you are starved for data, so memory bandwidth limits you. A high AI means you do lots of arithmetic per byte, so you reuse data heavily, so compute throughput limits you. Every algorithm sits somewhere on this spectrum.

Examples sharpen this. A vector add *c[i] = a[i] + b[i]* reads 16 bytes (two doubles), writes 8 bytes, and does 1 FLOP. AI is 1 FLOP / 24 bytes = 0.04. Extremely low. Vector add is always memory-bound. A dense matrix multiply, by contrast, does 2N cubed FLOPs and, if blocked well, moves only O(N squared) bytes. AI grows with N. For large N, matmul is compute-bound, in principle.

The Two Ceilings

Now the machine. Two numbers describe its limits.

- **Peak compute (FLOPs/s):** cores times clock times FLOPs-per-cycle. For an 8-core 3 GHz CPU with AVX2 doing 16 double FLOPs per cycle per core (8-wide FMA), peak is 8 * 3e9 * 16 = 384 GFLOP/s.

- **Peak memory bandwidth (bytes/s):** a hardware spec, measurable with a STREAM benchmark. A typical dual-channel DDR4 desktop hits around 30 to 40 GB/s. A server with eight channels can hit 150 GB/s or more.

The roofline plot puts arithmetic intensity on the x-axis (log scale) and achievable performance on the y-axis (log scale, in FLOP/s). Draw two lines. A diagonal line, performance = bandwidth times AI, rising from the left: this is the memory-bandwidth ceiling, because if you are bandwidth-limited, more arithmetic intensity buys you more performance proportionally. A horizontal line at peak compute: this is the compute ceiling, which you cannot exceed no matter how high your AI. The two lines meet at a corner called the ridge point. Your program is a dot under this roof. If your dot is left of the ridge, you are in the memory-bound region. If right of the ridge, compute-bound.

> **And Here Is Where It Gets Beautiful**
> The roofline turns the vague question "is my code good?" into a precise one: how far is my dot below the roof, and which part of the roof is above me? If you are at sixty percent of the memory-bandwidth ceiling and your AI is below the ridge, you are memory-bound and your remaining headroom is forty percent. If you are at five percent of the ceiling, you have 20x headroom and should keep working. The roofline tells you both how much room is left and which direction (more arithmetic intensity, or more bandwidth utilization) to push.

Roofline for Naive Matmul

Let us compute the roofline position for our naive matmul, because the answer is surprising and instructive.

FLOPs: matmul is 2N cubed = 2 * 1024 cubed = about 2.15 billion FLOPs. DRAM bytes: in the ideal blocked case, you move the three matrices once, 3 * N squared * 8 = 3 * 1024 squared * 8 = 25 MB. That gives an ideal AI of 2.15e9 / 25e6 = about 85 FLOPs per byte. Eighty-five is a high arithmetic intensity, well to the right of the ridge point of most machines. So in principle, matmul should be compute-bound and run near peak FLOPs.

But naive matmul does not achieve that AI. Because of the stride-n access to B and the cache thrashing, naive matmul re-reads B from DRAM far more than once. Its *effective* DRAM traffic is much higher than the ideal 25 MB, perhaps by a factor of ten or more. So its *effective* arithmetic intensity collapses from 85 down to single digits, dragging the dot left into the memory-bound region. The roofline reveals the real story: naive matmul is a fundamentally compute-bound algorithm (high theoretical AI) that has been artificially turned memory-bound by bad cache behavior. The fix (loop reorder, tiling) is precisely about restoring the theoretical AI by stopping the redundant DRAM traffic.

Key Insight

This is the deepest lesson in the book so far. An algorithm has an inherent arithmetic intensity. A bad implementation throws that intensity away by causing redundant memory traffic. perf stat sees the symptom (low IPC, high miss rate). The roofline sees the cause (effective AI dragged left of the ridge). Optimization is the act of moving your dot back toward the algorithm's true arithmetic intensity and then up toward the roof.

The GPU Roofline for Dorado

The roofline applies identically to GPUs, with different numbers. Take the NVIDIA T4, a common inference GPU and likely what you will profile Dorado on. Its specs: about 65 TFLOP/s of FP16 (tensor core), about 8 TFLOP/s FP32, and 320 GB/s of memory bandwidth. The ridge point for FP16 is at AI = 65e12 / 320e9 = about 203 FLOPs per byte. That is a very high ridge, which is the whole point of tensor cores: they make compute so cheap that almost everything becomes memory-bound unless arithmetic intensity is enormous.

Dorado's Transformer attention is the kernel to watch. Attention has moderate arithmetic intensity that depends on sequence length and head dimension. For the sequence lengths in basecalling, attention often lands left of the T4 ridge, meaning it is memory-bandwidth-bound, spending its time moving the key-value tensors rather than doing matmuls. That is the hypothesis. Nsight Compute (Chapter 13) will place Dorado's kernels on the GPU roofline and tell us, for each kernel, how far below the roof it sits and whether it is bound by HBM bandwidth or by tensor-core throughput. The answer drives the optimization: a memory-bound attention kernel is helped by caching and fusion (the Signal-to-Base cache idea), not by more compute.

How This Connects to Your Project

The roofline is what lets the two-page report make a defensible claim about how good the current performance is. For Kraken-2: compute its effective arithmetic intensity, plot it on the CPU roofline, and report "Kraken-2 achieves X% of peak DRAM bandwidth at AI of Y, confirming it is memory-bandwidth-bound." For Dorado: use Nsight Compute's built-in roofline section to report "the attention kernel achieves Z% of T4 peak and sits left of the ridge, confirming a memory-bandwidth bottleneck."

Without the roofline, the report can only say "the code is slow." With the roofline, the report says "the code is at 18% of its hardware ceiling and is memory-bound, so the right optimization is to reduce memory traffic, not to add compute." That sentence is the difference between a profile report and a profile report that recommends the correct fix.

5.5 perf bench: Measuring the Machine Itself

To draw a roofline you need the machine's actual peak memory bandwidth, not the marketing number from the spec sheet. perf ships with built-in microbenchmarks that measure it directly.

```bash
perf bench mem memcpy        # measure memcpy bandwidth
perf bench mem memset        # measure memset bandwidth
perf bench mem all           # run the whole memory suite
```

These give you the realistic sustained bandwidth your machine delivers, which is the y-intercept slope of your roofline's memory ceiling. You will typically see 60 to 80 percent of the theoretical DDR bandwidth, because real access patterns never hit the theoretical maximum. Use the measured number, not the spec number, when you compute what fraction of peak your program achieves. Reporting against the spec number understates your efficiency; reporting against the measured sustainable number is honest.

perf bench also has scheduler and futex benchmarks (*perf bench sched*, *perf bench futex*) for measuring synchronization and context-switch costs, which matter for the OpenMP version and for any heavily threaded workload like Kraken-2 at high thread counts.

5.6 Putting the perf Toolkit Together

We have now covered the entire perf toolkit. Let me give you the mental decision tree, because perf has many subcommands and knowing which to reach for is half the skill.

- **perf stat** — first contact. What is the IPC and cache miss rate? Compute-bound or memory-bound?
- **perf record + report** — where is the time? Which functions dominate?
- **perf annotate** — which instruction inside the hot function?
- **perf record -e cache-misses** — where do the cache misses originate?
- **perf mem** — which memory level satisfies each access, and at what latency?
- **perf c2c** — is there false sharing between threads?
- **perf diff** — what changed between two runs?
- **perf bench** — what is the machine's actual peak, for the roofline?

On Kraken-2 the sequence is: perf stat (find it is memory-bound), perf record (find the hash lookup is the hotspot), perf record -e cache-misses (confirm the lookup generates the misses), perf bench mem (measure peak bandwidth), then compute the roofline position (confirm it is bandwidth-bound and quantify the headroom), and finally, after adding the Hot-K-mer cache, perf diff (quantify the improvement). That is six perf commands and you have the entire Kraken-2 page of the report. No other tool needed for the CPU side.

What We Just Learned

perf diff compares two runs and shows the per-function delta. It is the tool that quantifies before-and-after for optimizations and catches regressions.

perf mem shows the memory hierarchy level and latency of each access. perf c2c finds false sharing, the cache-line ping-pong that makes parallel code slower than serial.

The Roofline Model is the framework for how-much-faster. Arithmetic intensity (FLOPs per byte) decides whether you are memory-bound (left of the ridge) or compute-bound (right of the ridge).

Naive matmul has a high theoretical arithmetic intensity but bad cache behavior drags its effective AI left into the memory-bound region. Optimization restores the true AI by eliminating redundant DRAM traffic.

The roofline applies identically to GPUs. The T4's high tensor-core throughput pushes the ridge point very high, so most kernels including Dorado's attention are memory-bandwidth-bound.

perf bench mem measures the machine's actual sustained bandwidth, the honest denominator for the roofline. Always report fraction-of-peak against the measured number, not the spec number.

Before You Move On

You have finished Batch 1. You can now answer all four profiling questions for any CPU program: what (perf stat), where (perf record), how much faster (roofline), what changed (perf diff). You understand the memory hierarchy that drives all of it. This is a complete, professional CPU profiling skill set.

Batch 2 turns to the Valgrind suite: Cachegrind for exact cache simulation (essential on WSL2 where the LLC counters do not work), Callgrind for call graphs, Memcheck for correctness, Massif for heap growth, and gprof as the classic instrumentation profiler. These trade perf's zero overhead for exactness. You use them when you need precise counts rather than statistical samples, or when the hardware counters you need are simply unavailable. Onward.